**SIMATS ENGINEERING**

# ASSIGNMENT 5

## Evaluation of Cryptographic Hash Functions and Digital Signatures in Electronic Health Record Security

Course Outcomes: CO2, CO3
Bloom's Level: BL5 – Evaluate
SDGs: 3, 9 and 16

Implementation: EHR CryptoGuard
Academic Prototype using Python, FastAPI, Cryptography and a responsive web UI

Student Name: _____
Register No.: _____
Team Members: _____
Faculty: _____
Academic Year: 2026–27

## Executive Summary

This assignment develops and evaluates an end-to-end integrity and origin-authentication workflow for synthetic Electronic Health Records (EHRs). The implementation treats a medical record as a canonical digital document, computes a cryptographic digest, binds that digest to the healthcare provider through a digital signature, and verifies both the digest and signature at the receiving side. The prototype compares SHA-256, SHA-3-256 and BLAKE2b-256, and supports ECDSA over P-256 and Ed25519 for signatures.

The main design decision is to separate integrity evidence from identity evidence. A hash alone can reveal that a byte sequence changed, but it does not establish who produced the record. A digital signature over the digest adds signer authentication and supports non-repudiation when the private key is controlled and attributable to the signer. The implementation therefore performs two verification checks: the current record must reproduce the signed digest, and the current digest must validate under the signer's public key.

The prototype also includes a tamper simulator, runtime hash benchmark, role-aware audit entries, security-control mapping and a user-friendly dashboard. Tests executed in the project environment passed for all core cryptographic routines. In the demonstrated tampering case, changing the diagnosis field changed the SHA-256 digest from aca4a5fc1ec017fd95579b802d74377dae413f38618c599e851359b928ecdc30 to 15370b78ce51db11f8e1d39e0a94be81ee97fbd6dfdc14f8d125460a0d181399, and verification was rejected.

## 1. Problem Statement and Problem Formulation

A telemedicine healthcare provider exchanges diagnoses, prescriptions, laboratory summaries and related EHR documents among hospitals, physicians, laboratories and insurance portals. These documents can cross organizational and network boundaries. An attacker who modifies a prescription, diagnosis or laboratory value during transmission or storage could create a clinically unsafe record or a fraudulent claim. The system therefore needs a mechanism that can detect modification, identify the source of an approved document, preserve evidence for later audit and impose an authorization boundary around sensitive operations.

The problem is formulated as follows: given an EHR document M produced by an authorized provider S, construct a verification package containing a cryptographic digest H(M) and a digital signature SigSK(H(M)). At the receiving side, accept the document only when H(Mreceived) equals the signed digest and the signature verifies under the trusted public key PK. If any protected field changes, the digest comparison must fail and the document must be rejected.

## 2. Objectives and Expected Outcomes

- Compare modern 256-bit cryptographic hash functions using integrity, collision-resistance rationale, output size and local computational overhead.
- Implement digital signature creation and verification using ECDSA P-256 and Ed25519.

- Demonstrate a controlled medical-record tampering event and measure its effect on verification.
- Create an auditable end-to-end workflow for a physician-generated EHR document.
- Map cryptographic mechanisms to integrity, origin authentication, non-repudiation, access control and auditability.
- Evaluate trade-offs and recommend a practical protocol profile for multi-party health information exchange.
- Produce reproducible source code, tests, execution evidence and a concise security analysis aligned to CO2 and CO3.

## 3. Requirements, Constraints and Assumptions

| Category | Requirement / Assumption |
|---|---|
| Functional | Create synthetic EHR; canonicalize data; hash; sign; verify; tamper; benchmark; audit. |
| Security | Use modern cryptographic primitives; never use SHA-1 or MD5 for the proposed deployment. |
| Data privacy | Only synthetic demonstration data is included; no real patient identifier is required. |
| Interoperability | JSON is used as a simple application-layer document representation. |
| Key management | Prototype generates ephemeral keys for demonstration; production requires managed PKI/HSM lifecycle. |
| Performance | Benchmark values are machine-dependent and are used only as local comparative evidence. |
| Deployment | The prototype is academic and is not presented as a certified clinical system. |
| Constraint | The assignment focuses on integrity, authentication and non-repudiation rather than full EHR confidentiality. |

## 4. Application of Relevant Course Knowledge / Concepts

| Course concept | Application in the assignment |
|---|---|
| Asymmetric cryptography | Private key signs; public key verifies. This separates signing authority from verification. |
| Cryptographic hashing | A fixed-length digest acts as a sensitive fingerprint of the canonical EHR. |
| Digital signatures | ECDSA/Ed25519 bind the digest to a private signing key. |
| Authentication | Signature verification provides cryptographic evidence that the holder of the signing key approved the digest. |
| Integrity | Any modification to signed content changes its digest and causes rejection. |
| Non-repudiation | When identity proofing, certificate binding, key protection and audit controls are trustworthy, signatures provide evidence that is difficult for the signer to deny. |
| Access control | Roles such as physician, laboratory and insurer are represented in the workflow and audit records. |
| Auditability | Signed and rejected events are recorded with timestamp, algorithm and signature identifiers. |

## 5. Proposed Solution and Methodology

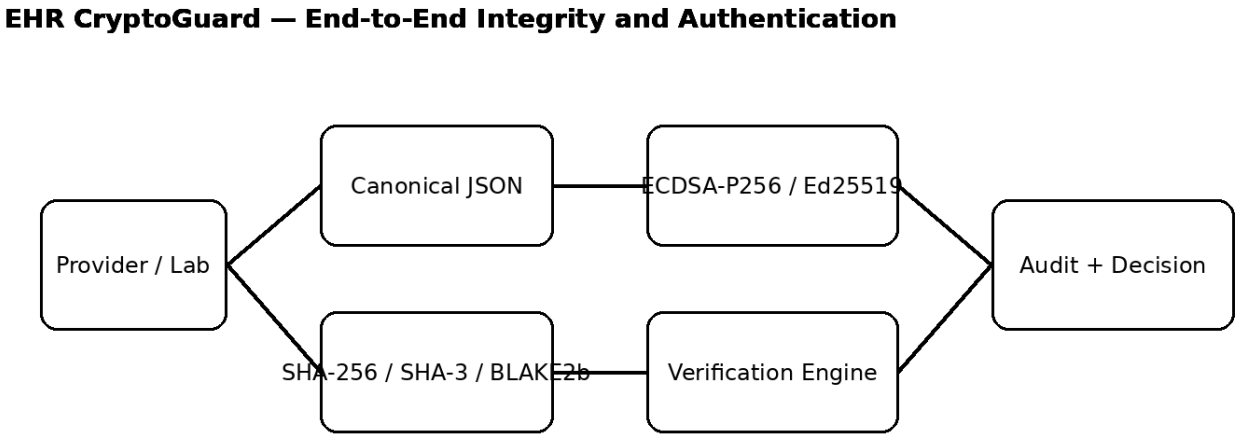


Figure 1. Proposed architecture. The sender creates a canonical document, computes a digest, signs the digest and forwards the record with verification evidence. The receiver independently recomputes the digest, checks the signature and records the decision.

### 5.1 Document Canonicalization

JSON objects do not guarantee a unique textual representation if field ordering or spacing differs. The prototype therefore serializes records with sorted keys, compact separators and UTF-8 encoding. This produces deterministic bytes before hashing. The canonical representation prevents an innocent formatting difference from appearing as a content modification.

```
canonical = JSON.stringify(record, sort_keys=True, compact_separators=True, UTF-8)
digest = HASH(canonical)
signature = SIGN(private_key, digest)
verification = (HASH(received_canonical) == signed_digest) AND VERIFY(public_key, signature, received_digest)
```

### 5.2 Hash Functions Evaluated

| Algorithm | Digest | Role in evaluation | Assessment |
|---|---|---|---|
| SHA-256 | 256 bits | Baseline widely deployed secure hash | Strong practical choice; excellent interoperability. |
| SHA-3-256 | 256 bits | NIST SHA-3 family alternative | Strong design diversity; useful where SHA-2 diversification is desired. |
| BLAKE2b-256 | 256 bits | High-performance modern hash | Very efficient in software; interoperability depends on application ecosystem. |

The evaluation does not claim that a local speed test proves global security. Collision resistance is a cryptanalytic property; the benchmark measures only implementation overhead on the local machine. The practical recommendation therefore weights security maturity and ecosystem support alongside measured runtime.

### 5.3 Signature Schemes Evaluated

| Scheme | Key property | Strengths | Trade-offs |
|---|---|---|---|
| ECDSA P-256 | Elliptic-curve signature | Strong ecosystem and PKI compatibility; compact public keys/signatures relative to RSA | Requires correct nonce handling and careful implementation; certificate lifecycle is important. |
| Ed25519 | EdDSA over Curve25519 family | Fast, compact, deterministic signing behavior and simple API | Certificate/enterprise interoperability may be less uniform than ECDSA in some legacy environments. |

## 6. Algorithm / Pseudocode / Flow

### 6.1 Signing Algorithm

INPUT: EHR record M, hash algorithm H, signing private key SK
1. C <- Canonicalize(M)
2. D <- H(C)
3. S <- Sign(SK, D)
4. Store/transmit {M, D, S, algorithm identifiers, signer metadata}
OUTPUT: signed EHR package

### 6.2 Verification Algorithm

INPUT: received record M', signed digest D, signature S, trusted public key PK
1. C' <- Canonicalize(M')
2. D' <- H(C')
3. hash_match <- (D' == D)
4. signature_valid <- Verify(PK, S, D')
5. ACCEPT only if hash_match AND signature_valid
6. Record result in audit log

### 6.3 Tamper Detection Logic

If attacker changes diagnosis/prescription/lab value:
canonical bytes change
digest changes
signed digest no longer matches
signature verification over the changed digest fails
decision = REJECTED

## 7. Implementation, Environment and Tools

| Item | Implementation |
|---|---|
| Language | Python 3.x |
| Backend | FastAPI + Uvicorn |
| Cryptography | Python cryptography package |
| Frontend | HTML5, CSS3 and vanilla JavaScript |
| Hashing | Python hashlib: SHA-256, SHA3-256, BLAKE2b-256 |
| Signatures | cryptography: ECDSA P-256 and Ed25519 |
| Testing | pytest |
| Packaging | ZIP source distribution with requirements.txt and README.md |
| Data | Synthetic EHR only |

The project is intentionally lightweight so that the cryptographic logic is visible and inspectable. The UI exposes the security operations rather than hiding them behind a large enterprise framework. This makes the demonstration suitable for a laboratory assignment and supports repeatable evaluation.

## 8. User Interface and Unique Demonstration Features

- Integrity Lab: a single workspace for loading a synthetic EHR, selecting a hash/signature pair and creating a signature.
- Tamper Simulation: changes the diagnosis field and immediately runs verification so the security effect is visible rather than only described theoretically.
- Dual Verification Evidence: displays both hash_match and signature_valid, helping distinguish content integrity from signature validity.
- Performance workspace: runs a fixed local benchmark across three 256-bit hashes.
- Audit Trail: records document-signing and verification decisions with timestamps and algorithm identifiers.
- Security Mapping view: maps each security requirement to the cryptographic or access-control mechanism and its observable evidence.
- Privacy-by-design demonstration: synthetic records are used, and the UI explicitly states that the prototype is not a clinical deployment.

## 9. Source Code Structure

```
EHR_Crypto_Assignment5/
├── app/
│   ├── main.py            # FastAPI routes and audit workflow
│   └── crypto_engine.py   # hashing, key generation, signatures, verification, benchmark
├── static/index.html      # responsive dashboard UI
├── tests/test_crypto.py   # cryptographic unit tests
├── screenshots/           # execution evidence
├── docs/                  # report diagrams and benchmark chart
├── requirements.txt
└── README.md
```

## 10. Test Cases and Expected / Actual Results

| ID | Test case | Expected result | Actual result |
|---|---|---|---|
| TC01 | SHA-256 digest length | 64 hexadecimal characters / 256 bits | PASS |
| TC02 | SHA-3-256 digest length | 64 hexadecimal characters / 256 bits | PASS |
| TC03 | BLAKE2b-256 digest length | 64 hexadecimal characters / 256 bits | PASS |
| TC04 | ECDSA sign then verify unchanged record | Verification = true | PASS |
| TC05 | Ed25519 sign then verify unchanged digest | Verification = true | PASS |
| TC06 | Alter diagnosis after signing | Hash mismatch and signature rejection | PASS |
| TC07 | Run 300 hash operations | Runtime metrics produced | PASS |

| | | | |
|---|---|---|---|
| | per algorithm | | |
| TC08 | Audit event creation | Signing/verification events recorded | PASS |

Automated project tests: 3/3 tests passed in the execution environment. The web workflow was also exercised through the API: signing produced a valid SHA-256 digest and ECDSA P-256 signature; altering the diagnosis caused both the hash-match check and signature verification check to return false.

## 11. Execution Screenshots / Outputs

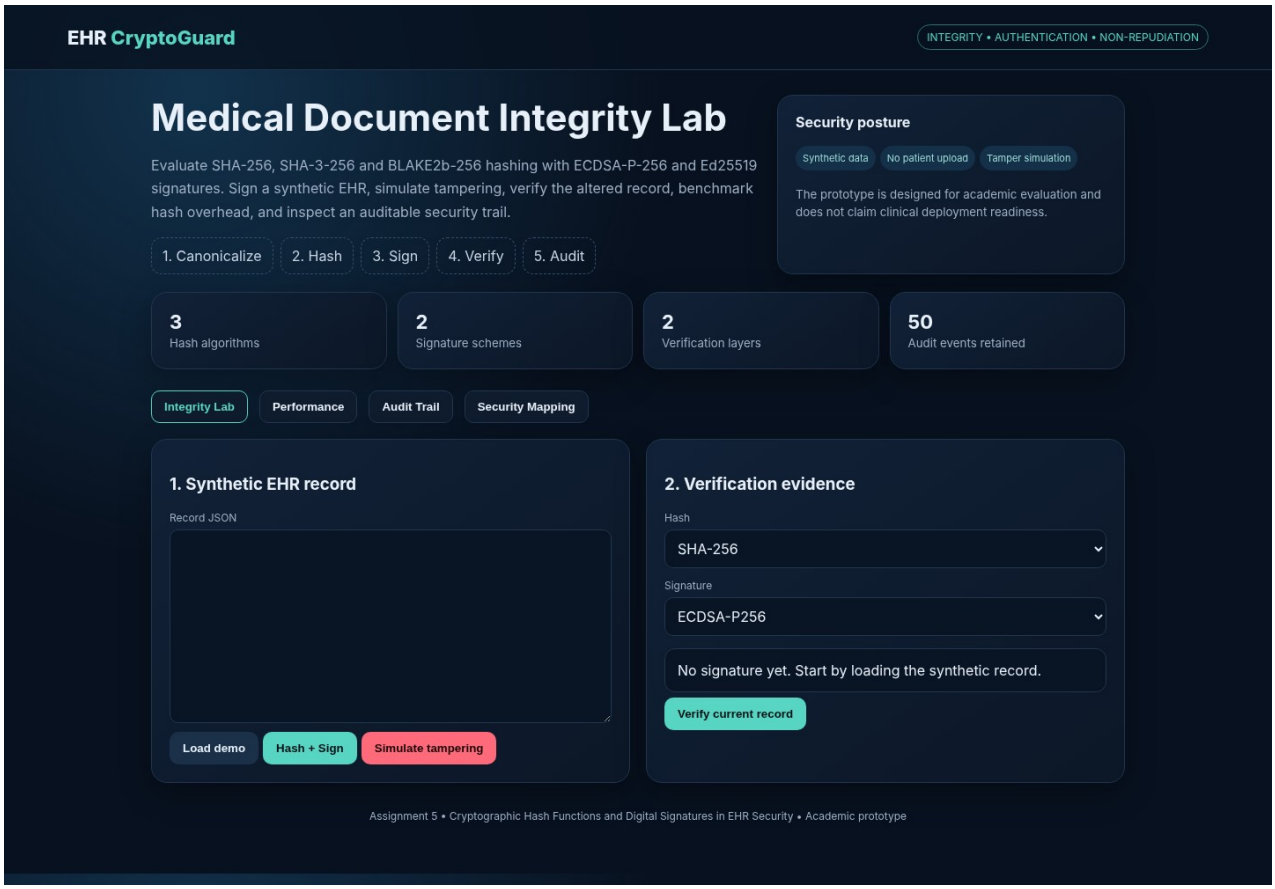


Figure 2. EHR CryptoGuard dashboard.

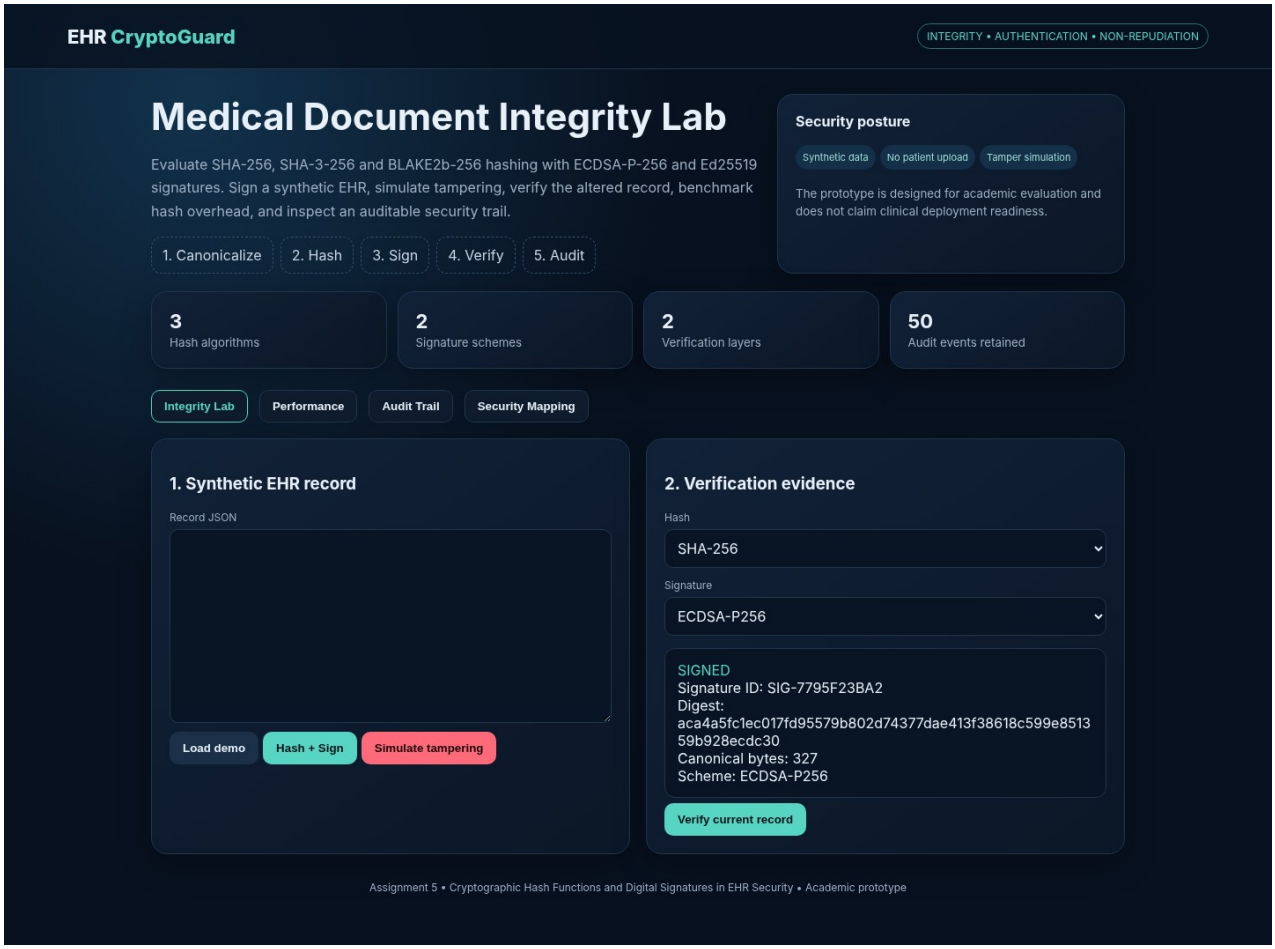


Figure 3. Signed-document state with digest and signature metadata.

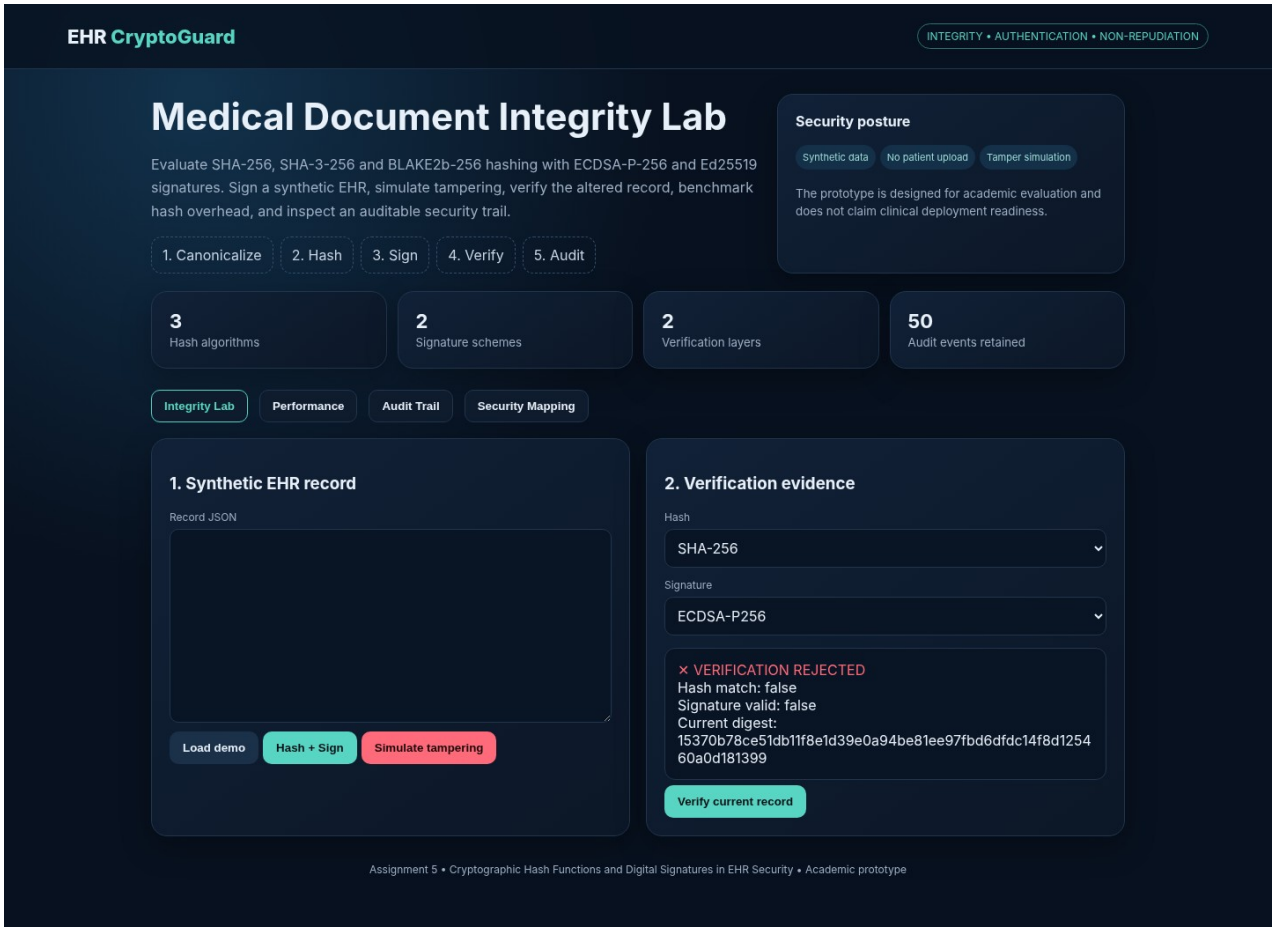


Figure 4. Tampered diagnosis rejected by verification.

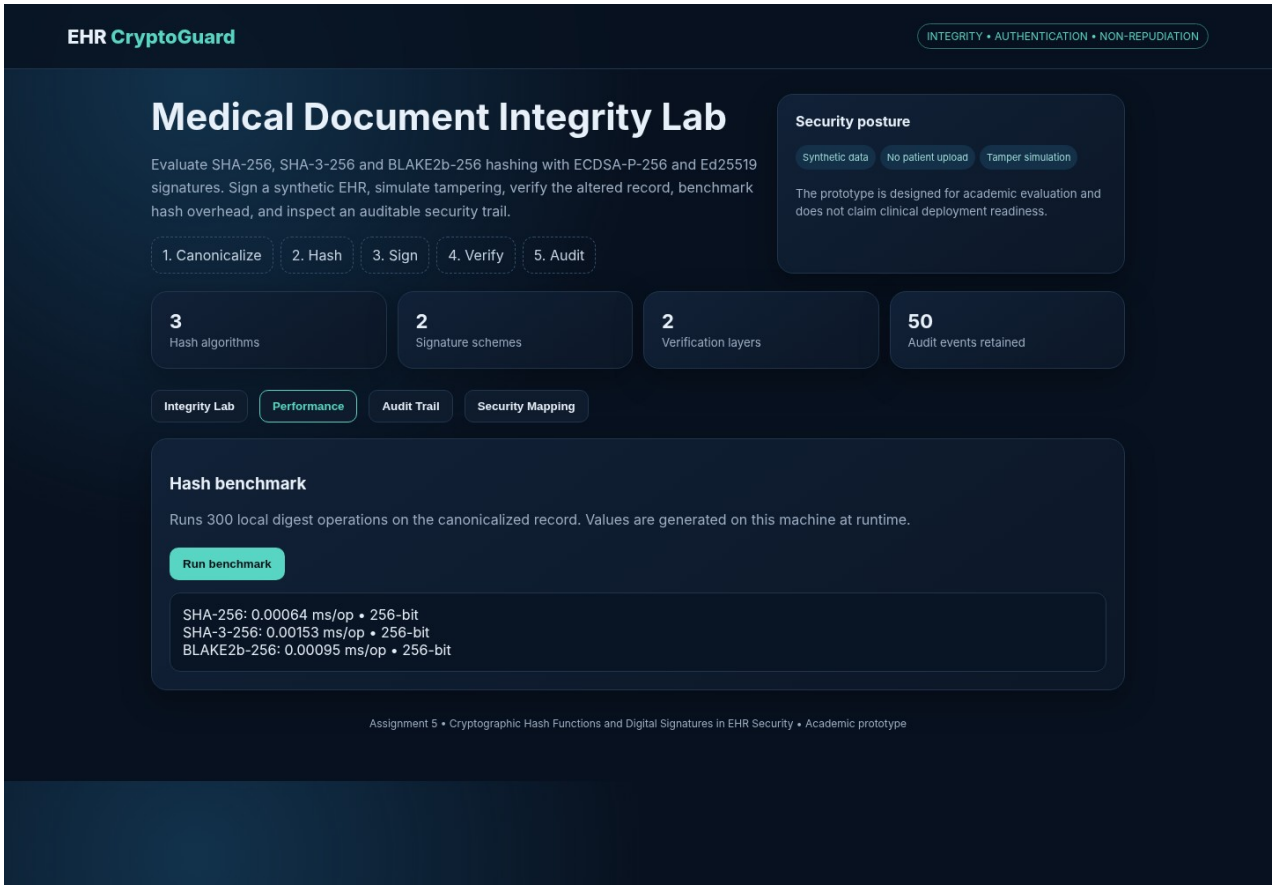


Figure 5. Hash performance workspace.

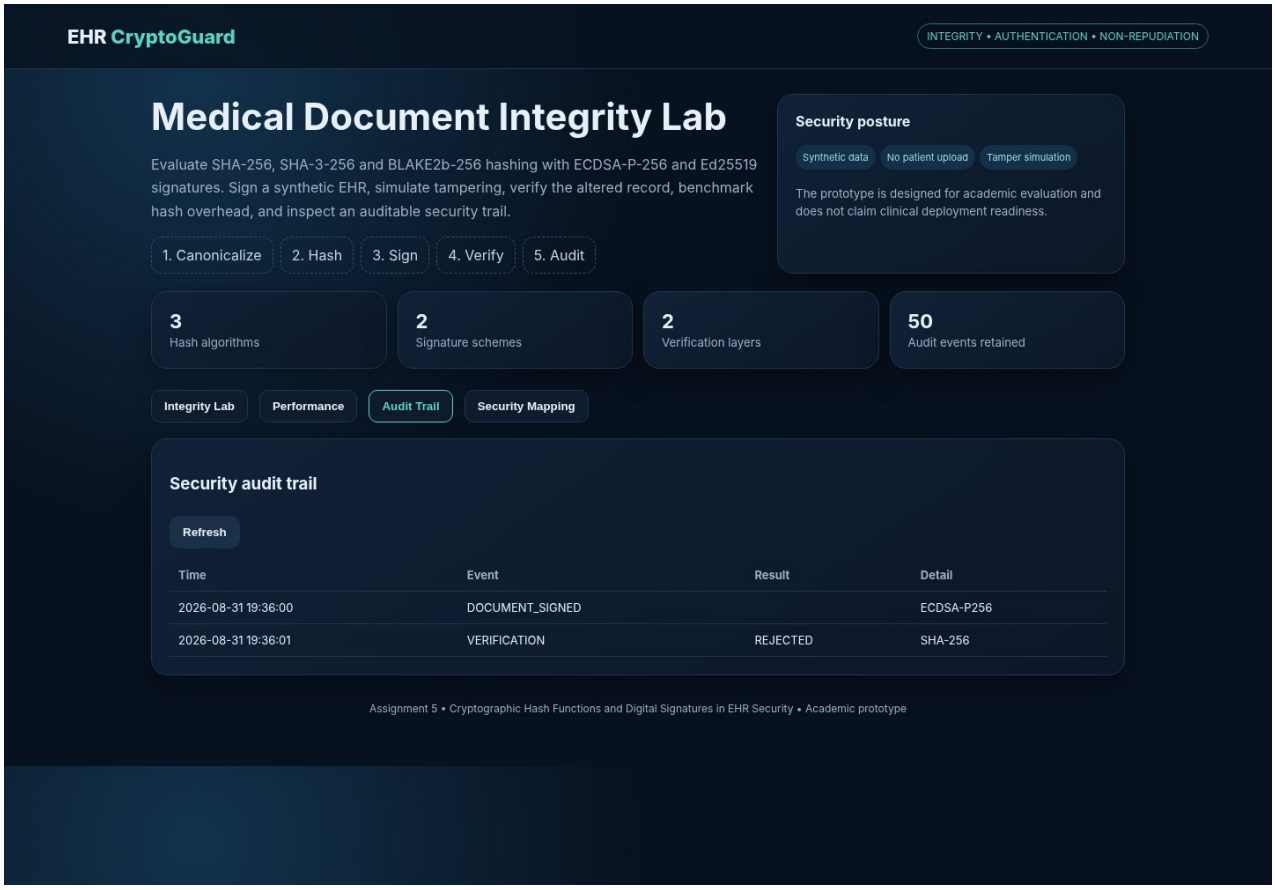


Figure 6. Audit trail workspace.

## 12. Experimental Results and Performance Analysis

| Hash | Rounds | Total time (ms) | Average (ms/op) | Output |
|---|---|---|---|---|
| SHA-256 | 300 | 0.192 | 0.00064 | 256-bit |
| SHA-3-256 | 300 | 0.458 | 0.00153 | 256-bit |
| BLAKE2b-256 | 300 | 0.284 | 0.00095 | 256-bit |

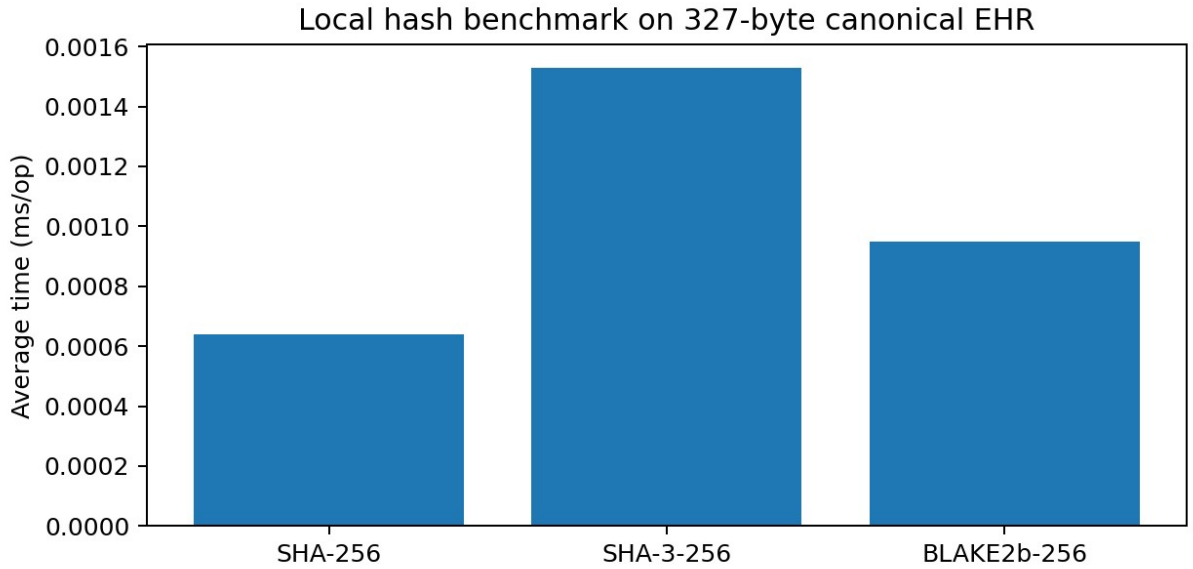

Figure 7. Local benchmark generated from the implemented 327-byte synthetic EHR record. Results are illustrative of the execution environment and should not be generalized to server, mobile, HSM or production workloads.

For this run, SHA-256 was the fastest of the three algorithms, BLAKE2b-256 was intermediate, and SHA-3-256 had the highest average runtime. The differences are extremely small at the 327-byte record size. Consequently, protocol selection should not be based on this micro-benchmark alone. SHA-256 is recommended for the primary deployment profile because it combines modern security with broad interoperability; SHA-3-256 can be retained as a diversification option; BLAKE2b-256 is attractive where the application ecosystem already standardizes it.

## 13. Security Evaluation Against Assignment Criteria

| Criterion | Evaluation | Evidence / judgement |
|---|---|---|
| Integrity assurance | High | Any protected content change produces a different digest; tampering test was rejected. |
| Authentication capability | High | Signature verification binds the digest to the corresponding public key. |
| Non-repudiation | Conditional / high with PKI | Requires trusted identity proofing, private-key protection, certificate status and audit controls in production. |
| Collision resistance | High for selected 256-bit modern hashes under current practical assumptions | No collision attack is attempted; security is assessed from established algorithm properties. |
| Computational overhead | Low for document-sized payloads | All three hashes completed 300 rounds in sub-millisecond total time on the test machine. |
| Cryptographic resilience | High when current parameters and libraries are maintained | Avoid deprecated SHA-1/MD5; maintain key rotation, certificate revocation and library updates. |
| Auditability | Good prototype evidence | Events include operation, algorithm, signature ID/result and timestamp. |
| Access control | Prototype-level | Role metadata is present; production needs centralized RBAC/ABAC and policy enforcement. |

## 14. Comparison, Trade-offs and Final Justification

| Option | Integrity | Interoperability | Performance in test | Recommendation |
|---|---|---|---|---|
| SHA-256 + ECDSA P-256 | Strong | Excellent | Fastest hash in test | PRIMARY |
| SHA-3-256 + ECDSA P-256 | Strong | Very good | Slower hash in test | SECONDARY / diversification |
| BLAKE2b-256 + Ed25519 | Strong | Good but ecosystem-dependent | Fast | SPECIALIZED / controlled ecosystem |

The recommended academic deployment is SHA-256 + ECDSA P-256 with a managed certificate authority and protected signing keys. The reason is not that ECDSA is universally superior to Ed25519; rather, ECDSA P-256 provides a practical bridge to certificate-based multi-organization environments. Ed25519 is retained as an attractive alternative for systems where the surrounding infrastructure supports it. In a production EHR exchange, the signature key should be distinct from encryption keys, and the signing identity should be backed by a trustworthy PKI or equivalent identity framework.

## 15. Security Architecture Improvements for a Production EHR Network

- Use TLS 1.3 for transport protection; the digital signature is an application-level authenticity/integrity layer, not a replacement for secure transport.
- Use X.509 certificates or an equivalent trust framework to bind provider identity to public keys.
- Protect private signing keys using an HSM or equivalent hardware-backed mechanism and enforce key rotation/revocation procedures.
- Keep signing and encryption key pairs separate so compromise of one purpose does not automatically compromise the other.
- Use RBAC/ABAC for physician, laboratory, insurer and administrator privileges, with least privilege and purpose limitation.
- Write security audit events to append-only or tamper-evident storage and restrict audit modification privileges.
- Use timestamping and certificate-status evidence where long-term verification and legal evidentiary requirements apply.
- Keep patient content off public ledgers. If blockchain is considered, store minimal commitments or audit evidence rather than unnecessary clinical content.
- Add secure backup, disaster recovery, key escrow/recovery policy and incident-response procedures.

## 16. Broader Considerations and SDG Relevance

| SDG | Connection to the implementation |
|---|---|
| SDG 3 – Good Health and Well-being | Integrity protection reduces the risk that a prescription, diagnosis or laboratory value is silently altered in a digital care workflow. |
| SDG 9 – Industry, Innovation and Infrastructure | The prototype demonstrates a modern cryptographic infrastructure pattern for interoperable health information exchange. |
| SDG 16 – Peace, Justice and Strong Institutions | Auditable signatures, accountable identities and tamper-evident records support institutional trust and evidence-based governance. |

## 17. Limitations

- The prototype does not implement a full healthcare interoperability standard such as FHIR resource validation.

- Keys are generated in the application for demonstration; production key custody requires HSM/PKI integration.
- The audit trail is in application memory and is not a durable tamper-resistant ledger.
- No real certificate revocation, OCSP/CRL workflow or trusted timestamp authority is implemented.
- Performance measurements are local micro-benchmarks and do not represent clinical production infrastructure.
- The prototype addresses integrity/authentication more directly than confidentiality and availability.
- No claim is made that the prototype is compliant with HIPAA, GDPR, Indian DPDP Act requirements or medical-device regulations; compliance needs legal, organizational and technical controls beyond cryptography.

## 18. Possible Improvements

- Add FHIR JSON canonicalization and resource-level signature support.
- Integrate a certificate authority and revocation checking.
- Add HSM-backed signing and key rotation.
- Add role/attribute-based policy enforcement with explicit consent and emergency-break-glass workflows.
- Add encrypted storage and envelope encryption for confidentiality.
- Add durable append-only audit storage and external trusted timestamps.
- Expand benchmarking to different EHR sizes, concurrent requests and realistic network latency.
- Add formal threat modeling using STRIDE or attack trees and penetration testing of the web API.

## 19. Individual Contribution of Group Members

| Member | Contribution | Evidence |
|---|---|---|
| Member 1 – _____ | Cryptographic engine, hashing and signature routines | crypto_engine.py; unit tests |
| Member 2 – _____ | FastAPI backend, verification workflow and audit API | app/main.py |
| Member 3 – _____ | UI design, tamper simulation, benchmark and screenshots | static/index.html; screenshots |
| Member 4 – _____ | Research review, report analysis, SDG/CO mapping | report and references |

If the team size differs, replace the rows with the actual individual contributions. Each member should retain evidence of their work, such as commits, test results, screenshots or report sections.

## 20. One-Page Individual Reflection

Working on this assignment changed my understanding of integrity from a simple checksum idea into a security property that depends on both the cryptographic primitive and the surrounding trust model. I first treated the hash as the main solution, but the implementation made the distinction clear: a hash can detect a change only when the expected digest is trusted, while a digital signature

also provides evidence connected to a signing key. For that reason, the final workflow uses a hash followed by a digital signature and verifies both conditions at the receiver.

A major design decision was to use canonical JSON before hashing. Without canonicalization, two semantically identical records could produce different byte strings because of formatting or field-order differences. I also added a tamper simulator because it gives a stronger demonstration than showing a successful verification alone. When the diagnosis field was changed, the digest changed and the signature verification failed. This was the most useful experimental observation because it connected the mathematical idea of hashing with a realistic EHR security event.

The main challenge was balancing security features with a manageable assignment implementation. I therefore kept the cryptographic core small and visible, while adding a user-friendly interface for signing, verification, benchmarking and audit review. I learned that non-repudiation cannot be claimed from an algorithm in isolation. It depends on identity proofing, private-key protection, certificate lifecycle management, reliable timestamps and audit controls. This is an important distinction for a healthcare environment where evidence may need to remain trustworthy long after a document was created.

The assignment also helped me connect CO2 and CO3. CO2 is reflected in the asymmetric architecture: a private key is used for signing while the public key supports independent verification. CO3 is reflected in the comparison of hash algorithms, digital signatures, access-control roles and audit evidence. The work aligns with SDG 3 because protecting the integrity of medical information can reduce risks caused by corrupted digital records; with SDG 9 because secure cryptographic infrastructure supports digital healthcare systems; and with SDG 16 because accountable signatures and audit trails strengthen trust and institutional responsibility.

If I extend the project, I would integrate FHIR resources, a real certificate authority, hardware-backed key storage and durable tamper-evident audit logging. I would also test larger datasets and concurrent users rather than relying only on a local micro-benchmark. Overall, the assignment gave me practical experience in evaluating security mechanisms rather than simply implementing an algorithm, which is directly aligned with the BL5 evaluate level.

## 21. Conclusion

The EHR CryptoGuard prototype demonstrates that a modern hash function and digital signature scheme can be combined into an effective document-integrity and origin-authentication workflow. The implementation successfully hashes, signs and verifies synthetic medical records, detects an unauthorized modification to the diagnosis field, benchmarks three modern hash functions and preserves a prototype audit trail. The experimental run showed SHA-256 as the fastest hash among the tested options for the small record size, while all selected algorithms produced 256-bit digests. The final recommendation is SHA-256 with ECDSA P-256 for a broadly interoperable deployment profile, with SHA-3-256 and Ed25519 retained as strong alternatives where ecosystem requirements justify them.

The key conclusion is that cryptography is necessary but not sufficient for trustworthy EHR exchange. Production security must combine cryptographic primitives with identity management, access control, key custody, revocation, secure transport, audit protection, privacy controls and operational governance. The assignment therefore provides a defensible academic prototype and a practical security architecture that can be extended toward a standards-based healthcare environment.

## 22. References – Recent Research and Standards


1. Winter, M., Kraft, R., Leber, P., Reichert, M., Greger, H., & Muhr, J. (2026). Cybersecurity in eHealth: A Scoping Review of Current Research and Trends. IEEE Journal of Biomedical and Health Informatics. DOI: 10.1109/JBHI.2026.3687103.
2. EHRAuditChain (2026). EHRAuditChain: Scalable privacy-preserving EHR audit with RSA accumulators on blockchain. Information Sciences, 736, 123109. DOI: 10.1016/j.ins.2026.123109.
3. MeDiStore trust protocol integrating reputation based proof of stake consensus for enhanced security of blockchain networks in electronic health record management (2025). Discover Computing. DOI: 10.1007/s10791-025-09540-2.
4. Lee, C. H., Lim, K. H., & Eswaran, S. (2025). A comprehensive survey on secure healthcare data processing with homomorphic encryption: attacks and defenses. Discover Public Health, 22, 137. DOI: 10.1186/s12982-025-00505-w.
5. A Hybrid Secure Signcryption Algorithm for data security in an internet of medical things environment (2024). Journal of Information Security and Applications, 85, 103836. DOI: 10.1016/j.jisa.2024.103836.
6. ECDSA-based tamper detection in medical data using a watermarking technique (2024). International Journal of Cognitive Computing in Engineering, 5, 78–87. DOI: 10.1016/j.ijcce.2024.01.003.
7. A novel medical steganography technique based on Adversarial Neural Cryptography and digital signature using least significant bit replacement (2024). International Journal of Cognitive Computing in Engineering, 5, 379–397. DOI: 10.1016/j.ijcce.2024.08.002.
8. A highly secured EHR management system based on blockchain technology with digitally signed authentication using data sanitization and polynomial interpolation (2024). Biomedical Signal Processing and Control, 87, 105412. DOI: 10.1016/j.bspc.2023.105412.
9. A hybrid encryption algorithm based approach for secure privacy protection of big data in hospitals (2024). Egyptian Informatics Journal, 28, 100569. DOI: 10.1016/j.eij.2024.100569.
10. Secure Data Transmission of Electronic Health Records Using Blockchain Technology (2023). Electronics, 12(4), 1015. DOI: 10.3390/electronics12041015.
11. NIST. FIPS 180-4. Secure Hash Standard (SHS). National Institute of Standards and Technology.
12. NIST. FIPS 186-5. Digital Signature Standard (DSS). National Institute of Standards and Technology.
13. Python Cryptography Project documentation. Cryptographic recipes and primitives used by the implementation.
14. FastAPI documentation. Python web framework used for the academic prototype.

## Appendix A – Demonstration Record


```
{
  "patient_id": "EHR-DEMO-001",
  "patient_name": "Asha Raman",
  "date": "2026-08-31",
  "provider": "Dr. Meera Iyer",
  "facility": "Sentinel Telehealth Centre",
  "diagnosis": "Acute respiratory infection",
  "prescription": "Amoxicillin 500 mg — as prescribed",
  "lab_summary": "CRP mildly elevated; oxygen saturation 97%",
  "sensitivity": "RESTRICTED"
}
```


## Appendix B – Reproducibility Checklist

- Create a Python virtual environment.
- Install requirements.txt.
- Run uvicorn app.main:app --reload.
- Open the local dashboard.
- Load demo record.
- Sign with SHA-256 + ECDSA-P256.
- Verify unchanged record and observe PASS.
- Simulate tampering and observe REJECTED.
- Run benchmark and record local values.
- Run pytest and confirm all tests pass.
- Zip the project folder for GitHub or LMS submission.